A Practical Hybrid Robust VAE-CatBoost Framework for Efficient Multiclass Network Intrusion Detection

Vihaan Ovalekar(✉), Ojas Gore, Yash Pradhan, Ishan Tanawade, and Deepali Patil

Department of Artificial Intelligence and Data Science, SVKM's Dwarkadas J. Sanghvi College of Engineering, Mumbai, India
vihaan.ovalekar@gmail.com, ojasgore2005@gmail.com, yashpradhan0712@gmail.com, ishanst995@gmail.com

Abstract. Machine-learning intrusion detectors often report detection rates near 100 %, but random splits and shortcut features inflate such results, and the detectors miss attack families absent from training. We propose a hybrid framework in which a robust variational autoencoder (VAE), with a Student-t likelihood, missing-entry handling of undefined flow statistics and a topological loss, gives a CatBoost classifier its inputs and gives two streaming components their signals. From the reconstruction error and latent space of the VAE, a correction relabels bursts of novel flows with the dominant recent attack class, and a detector flags unseen attack families. On CSE-CIC-IDS2018 with a time-ordered hold-out and three seeds, the framework reaches a mean F1 of 0.991 ± 0.004 over the four larger classes, against 0.979 ± 0.001 for raw-feature CatBoost and 0.975 for the best of ten re-run methods. The gain comes from the correction on the VAE signals, which also lifts raw-feature CatBoost to 0.990. With the detector, trained on attacks of other days, the framework has the highest Youden's J on left-out days (30.27 against at most 24.57), and the detector alone finds 30.86 % of the flows of left-out families, against 4.17 % on raw features. The framework needs 0.05 ms per flow on one CPU thread.

Keywords: Intrusion detection · Variational autoencoder · Student-t likelihood · CatBoost · Novelty · Unseen attack families

1 Introduction

Intrusion detection systems (IDS) that learn from network flows must handle class imbalance, heavy-tailed flow statistics and attacks that were absent from training. Hybrid pipelines of feature selection, autoencoders and gradient-boosted trees report recalls near 99 % [1] and accuracies near 98 % [2] on public flow datasets, but such numbers can be optimistic: random splits put flows of the same attack session into training and test data, identifier columns such as ports let a model learn the attack machines, and plain accuracy hides the small classes [3–6]. Moreover, a classifier can only output the classes that it has seen, so its detection can collapse for a new attack family [7, 8].

This paper keeps the hybrid design but changes what the autoencoder is for. In our framework (Fig. 1), CatBoost with SHAP importance selects the features (Stage 1), a robust variational autoencoder (VAE) encodes them (Stage 2), and a CatBoost classifier

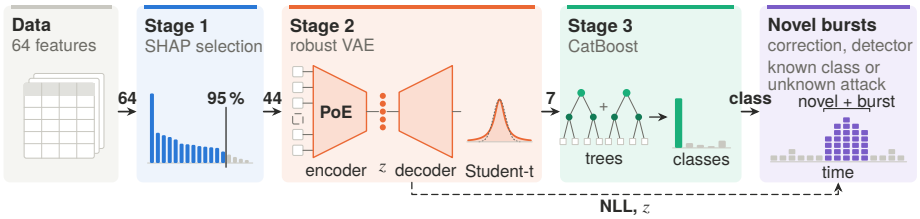


Fig. 1. Architecture of the framework: Stage 2 encodes the defined features with a product of per-feature experts (PoE; the expert of an undefined statistic, dashed, is left out). From the VAE NLL and the latent mean z, the correction relabels bursts of novel flows (marked novel + burst); the detector checks the remaining Benign flows. Both belong to Stage 3.

predicts the class from the VAE outputs (Stage 3). The VAE outputs also show how unusual a flow is, and its latent space groups similar flows; Stage 3 uses these signals in a streaming correction and a detector of unseen attacks. We call Stages 1–3 without the detector *the framework*, and write *with the detector* when it is added. Our ablations show that this second role of the VAE carries the gain. Our contributions are:

1. A robust VAE for CICFlowMeter flow features, which treats undefined statistics as missing entries instead of zeros, models heavy tails with a Student-t likelihood, and adds a topological loss for latent neighbourhoods (Sect. 3).
2. A streaming correction that relabels bursts of novel flows, and a monotone CatBoost detector of unseen families; both run online without host addresses (Sect. 3).
3. An evaluation on CSE-CIC-IDS2018 on a hold-out, left-out days, leave-one-attack-out folds and a test day, against ten re-run methods, with ablations and CPU timing (Sect. 4).

2 Related Work

Detection varies widely, and can collapse, when the attack classes of the test data are absent from training [7–10]. Earlier hybrids feed autoencoder errors [1] or VAE features [2] to tree classifiers. Streaming detectors keep per-host statistics [11], score bursts of edges between hosts [12] or classify windows of flows [13]. Our streaming components instead count bursts in the latent space of the VAE, so they need no host addresses.

3 Proposed Framework

3.1 Stage 1: CatBoost with SHAP Feature Selection

Stage 1 ranks the 64 candidate features by their mean absolute SHAP value [14] over 20 CatBoost runs, each on a random 70 % of the features, and keeps the smallest top set that holds 95 % of the importance: 44 features on our data (Fig. 2, Table 1).

3.2 Stage 2: Robust Variational Autoencoder

Each part of our VAE [15] targets one property of flow statistics.

Undefined Statistics. CICFlowMeter writes 0 when it cannot compute a statistic [16]: for example, an inter-arrival time needs two packets and an idle time needs an idle period. A zero of this kind is not a measurement. In the training rows, 81.04 % of the *Idle Max* values and 28.82 % of the *Flow IAT Std* values are undefined. We mark these entries as missing ($b_j = 0$) with rules from the CICFlowMeter source code. Each feature j has its own small encoder, an expert, that maps x_j to a Gaussian $\mathcal{N}(\mu_j, \sigma_j^2)$ over the latent vector $z \in \mathbb{R}^5$. The posterior is the product of the experts of the defined features and a standard normal prior expert [17]: $\sigma^{-2} = 1 + \sum_j b_j \sigma_j^{-2}$ and $\mu = \sigma^2 \sum_j b_j \sigma_j^{-2} \mu_j$. A missing feature thus has no effect on the posterior and adds no likelihood term [18, 19].

Heavy Tails. In the training rows, flow durations span six orders of magnitude and forward packet counts four, and a Gaussian likelihood lets a few extreme flows dominate the loss. The decoder therefore gives each feature a Student-t likelihood with learned location, scale and degrees of freedom [20], whose negative log-likelihood we call ℓ_j. A soft lower bound on the log scale prevents the scale from collapsing [21].

Neighbourhoods. The correction of Stage 3 counts flows in cells of the latent space, so similar flows must stay close after encoding. The VAE objective does not ensure this. We therefore add the 0-dimensional topological loss $\mathcal{L}_t$ of Moor et al. [22], which compares the minimum spanning trees of the inputs and of the posterior means within a batch.

Objective. To train the experts for incomplete rows, each batch also contains a second view x_s of each row, in which a random subset s of the defined features is kept [18, 19]. The VAE minimizes

$$\mathcal{L} = \sum_{v \in \{x, x_s\}} \Big(\mathbb{E}_{q(z|v)} \big[\textstyle\sum_j m_j^v \ell_j \big] + \beta \, \mathrm{KL}\big(q(z \mid v) \,\|\, p(z)\big) \Big) + 0.1\beta \, \mathrm{KL}\big(q(z \mid x_s) \,\|\, q(z \mid x)\big) + \gamma \mathcal{L}_t, \quad (1)$$

where m_j^v is 1 for the features that view v contains, $p(z) = \mathcal{N}(0, I)$, and the third term makes the two views agree [23]. The weight is $\gamma = 1$ (Sect. 3.4).

3.3 Stage 3: Classification, Correction and Detection

Classifier. For each flow, the VAE gives seven outputs: the five posterior means μ, the KL term, and the reconstruction NLL at $z = \mu$, summed over the 23 features that every flow defines, so that the sum has the same terms for every flow. A multiclass CatBoost classifier [24] with balanced class weights learns the class from these outputs, and Optuna [25] tunes it. The classifier knows only the training classes, so in the tests on unseen families we count any class other than Benign as an attack, as in the binary view of [7].

Novel-Burst Correction. A learned attack can return in a form that the classifier misses: the teardown flows of a Slowloris attack look Benign to the classifier, but they are unusual for the VAE and arrive in bursts. The correction finds such flows in three steps. First, a flow is *novel* when its reconstruction NLL is above the q_n quantile of the NLL of the Benign flows of a calibration day. Second, a flow is part of a *burst* when many flows have recently arrived in its latent cell: after each latent axis is divided by its training SD, the cell is the grid cube of side w that contains the posterior mean of the flow. We measure this with the MIDAS-R score [12] and a threshold at its q_b quantile on Benign training flows, with the latent cell in place of the host pair because the flow records carry no addresses. Third, a novel flow in a burst takes the attack class that the classifier predicted most often among all earlier flows of the last W seconds of the day, if the count of that class passes a support threshold; otherwise its prediction stays. The calibration day contains none of the training attack classes, so every attack prediction on it is a false alarm, and the q_s quantile of this count over its Benign flows sets the threshold. All quantities use only earlier flows, so the correction runs online. In the hold-out evaluation, the training flows of the same days are part of the stream; they count as Benign predictions, so that no training label adds attack support.

Unseen-Attack Detector. A flow of a new attack family that resembles no learned class stays Benign after the correction, so a second CatBoost model, the detector, checks every Benign flow. Its first input is the *relative NLL*: the reconstruction NLL minus the running median NLL of the earlier non-burst flows of the same day, because days differ in their normal NLL level. The other four inputs are the KL term, the MIDAS-R score of the latent cell, the same score over novel flows only (relative NLL above its q_n quantile on calibration-day Benign flows), and the share of novel flows among the flows of the cell in the last W seconds. Each input is replaced by its percentile rank among Benign reference flows, so that days share one scale. Monotone constraints [26] let a larger input only raise the attack score, so a more unusual flow never gets a lower score and the detector cannot learn a narrow value range of one training family.

Its positives are labelled attacks of families that the encoding VAE and classifier never saw: the class left out in each leave-one-attack-out fold, encoded by the models of that fold, and the attacks of day 02-28 and of further capture days (Sect. 4.1). The negatives are one half of the Benign flows of these days and of the calibration day, and its cut-off gives 1 % false alarms on the other half. When we evaluate a left-out day or fold, that day with its families, or that fold class, is removed from the detector's training. A flow that it flags becomes an *unknown attack*, a wrong class in the seen-class metrics.

3.4 Hyperparameter Selection

No model weight or threshold is fitted on the hold-out or the test day. The topological weight γ and the five correction settings are selected with seed 3 on an inner split, which holds out the latest 20 % of the training rows of each day and class. The weight γ maximizes the mean per-class F1 with the correction at its start values. The correction settings are then set one at a time to maximize the mean attack-versus-Benign F1 on three leave-one-attack-out folds of the inner split. For the detector, the NLL is taken relative to either the median NLL of the calibration day or the running median of the day,

Table 1. Settings of the framework; Stage 3 values are search ranges of Optuna's TPE sampler.

Part	Settings
Stage 1	20 CatBoost runs on random 70 % feature subsets and ≤50,000 training rows per class, SHAP on 1,000 rows per class; 300 iterations, depth 6, learning rate 0.1, balanced class weights
Stage 2	all training rows; signed log, standardization on defined entries; 44 experts of 20 ReLU units, latent size 5; decoder 128 ReLU units, Student-t, soft bound $\log \hat{\sigma} \geq -6$, $\nu = 1 + \text{softplus}$; $\beta = 0.5$, consistency 0.1β; second view: each defined feature kept with probability $1 - \rho$, $\rho \sim U(0, 0.7)$, and, in addition, the features removed that are undefined in a random other row of the batch; $\gamma \in \{0.1, 0.3, 1, 3\}$, chosen $\gamma = 1$, on 256 flows per batch, distances divided by their mean; Adam $8 \cdot 10^{-3}$, batch 32,768, ≤1,000 epochs, learning rate halved after 3 flat epochs, stop after 10 epochs without a relative gain of 10^{-4} in the 02-28 ELBO
Stage 3	CatBoost, plain boosting, balanced weights; Optuna TPE, 20 trials (10 random); iterations 200–1000, depth 4–8, learning rate 0.01–0.3, L2 1–30 (both log scale); mean per-class F1 over 3 time-blocked folds of a 10 % sample per day and class (classes under 2,000 rows whole); same search for raw CatBoost; the repository gives the LightGBM and XGBoost search ranges
Correction	grids, searched in this order from the start values in brackets (ties keep the start value): $w \in \{0.1, [0.2], 0.5\}$, $q_b \in \{0.975, [0.99], 0.995\}$, $W \in \{30, [60], 120\}$ s, $q_s \in \{0.975, [0.99], 0.995\}$, $q_n \in \{0.90, 0.95, [0.975], 0.99\}$; chosen $w = 0.1$, $q_b = 0.975$, $W = 120$ s, $q_s = 0.975$, $q_n = 0.9$; latent axes divided by their training SD; MIDAS-R edge score only, decay 0.5 [12], one-minute ticks; flows of the same second are not earlier than each other
Detector	CatBoost, depth 2, 500 iterations, learning rate 0.1, monotone; rank references: relative NLL and KL of calibration-day Benign flows, stream inputs of Benign training flows; median NLL of a reservoir sample of ≤20,000 earlier flows of the day with MIDAS-R below its 0.9 Benign training quantile, refreshed every 500 flows (calibration-day median for the first 1,000); ≤200,000 negatives per day; 1 % false alarms

and the inputs are one of five subsets; each configuration is validated by leaving out one further capture day or one fold at a time. The selected configuration, the running median with all five inputs, has the highest mean detection rate among those with at most 2 % mean false alarms. Table 1 lists the grids and the selected values.

4 Experiments and Results

4.1 Dataset and Protocol

We used the official CSV files of CSE-CIC-IDS2018 [27, 28]. We corrected their timestamps, which CICFlowMeter writes on a 12-hour clock [16], and removed rows with non-finite values. To avoid shortcuts, we dropped the timestamp [4, 6], the destination port [5] and three TCP header fields whose values identify the attacker machines on the training days (*Init Fwd Win Byts*, *Init Bwd Win Byts*, *Fwd Seg Size Min*; similar shortcuts are reported in [4, 6]), together with 10 constant columns; 64 candidate features remain. We also removed duplicate rows, rows of other days that equal training rows, and label conflicts, that is, identical feature vectors with different labels. The conflicts include 184,551 FTP brute-force rows, in line with the failed FTP attack in [29].

Table 2 shows the splits. The classes of the test day, 02-21, are absent from training, so this day tests unseen families. Its HOIC window contains a burst of 358,621 Benign flows of one template; Liu et al. [29] report no label problem for HOIC apart from a time offset, so we kept these flows as Benign. Outside this burst, the day has only 2,125 Benign flows. The detector also trains on further capture days on which at least six hours each hold at least 1 % of the day's Benign flows; this rule excludes the truncated file of day 02-16. On day 02-20, we removed 797 UDP flows labelled LOIC-HTTP inside the LOIC-UDP window, because LOIC-UDP is a test-day class.

Table 2. Data splits after cleaning: the hold-out contains the latest 20 % of each day and class.

Split (use)	Day(s)	Rows (class counts)
Training (Stages 1–3)	02-14, 02-15	1,039,918 (Benign 923,868; SSH 75,224; GoldenEye 33,066; Slowloris 7,730; FTP 30)
Hold-out (seen classes)	02-14, 02-15	259,983 (Benign 230,968; SSH 18,807; GoldenEye 8,267; Slowloris 1,933; FTP 8)
Validation (VAE stop, detector)	02-28	222,123 (Benign 211,538; Infiltration 10,585)
Calibration	02-23	705,316 (Benign 704,897; web attacks 419)
Test (unseen classes)	02-21	555,189 (Benign 360,746; HOIC 192,713; LOIC-UDP 1,730)
Further days (detector)	02-20, 02-22, 03-01, 03-02	6,659,249 (Benign 5,896,584; LOIC-HTTP 573,708; Bot 143,315; Infiltration 45,408; web brute force 147; XSS 64; SQL injection 23)

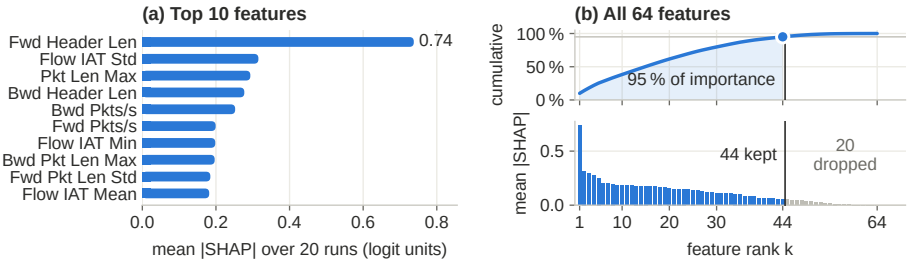


Fig. 2. Stage 1 SHAP ranking: (a) mean |SHAP| of the top 10 features over 20 runs; (b) cumulative share and importance of all 64 features (the 95 % rule keeps $k = 44$). The rankings of two disjoint halves of the runs have Spearman correlation 0.963 and top-44 Jaccard index 0.913.

Metrics and Runs. For seen classes, the main metric is the mean per-class F1 over the classes with at least 30 hold-out rows; FTP, with 8 rows, is reported separately. For unseen families, we report the detection rate (DR), the Benign false-positive rate (FPR), and Youden's $J = \mathrm{DR} - \mathrm{FPR}$. Each seed (3, 4, 5) retrains every model after Stage 1 and recomputes every threshold; Stage 1 runs once on the main split and once per fold. We report the mean and standard deviation (SD) over seeds and a paired bootstrap of the difference to raw CatBoost [30]. The VAEs and the PyTorch re-run methods were trained on an RTX 5080 Laptop GPU and the other models on an Intel Core Ultra 9 275HX CPU; the code repository gives the library versions, thread counts and the command for every table: https://github.com/WinterBlossom0/IDS-VAE-and-Catboost.

Comparison Methods. We re-ran ten methods from related work, the *re-run methods*, on the same features, rows and seeds (Table 3), with balanced class weights or class-balanced batches and no oversampling. Each method predicts the same hold-out, test day and further days, and is retrained without the left-out class in each fold.

4.2 Seen Classes: The Correction on VAE Signals Gives the Gain

We first ask whether Stage 2 and the correction improve on CatBoost with the same 44 features (raw CatBoost), on the four classes with at least 30 hold-out rows (Table 4).

Table 3. Re-run methods (top; Stage 1 features of the main split or fold; deviations from the source in italics; not given: absent from the source) and results of related work (bottom; other splits and metrics, so not comparable). LOAO: leave-one-attack-out

Method	Settings
Random forest [8]	scikit-learn defaults, as the source; standardization; *signed log, balanced class weights*
MLP [7]	24 hidden units (WEKA rule of the source code); *scikit-learn with early stopping, not WEKA*
XGBoost [31], LightGBM [32]	library defaults, signed log + standardization (XGBoost has no random step)
IDAE-XG [1]	as the source; top 22 Stage 1 features by random-forest rank and 20 autoencoder epochs (both not given); *multiclass, binary in the source; no upsampling*
CNN-1D [33]	as the source ($\leq$20 epochs, patience 4); *our split, stratified random split in the source*
FlowTransformer [13]	8-flow windows of the day (source code), 2 decoder blocks, learning rate from the source grid; *multiclass*
TabM [34]	package defaults, settings of the source (patience 16); *no tuning*
FT-Transformer [35]	default configuration and optimizer (patience 16); *no tuning, no ensemble*
TabNet [36]	Table 4 of the source, patience 20; learning rate 0.02 (library default; not given); *no SMOTE*
Work	**Data; split; reported result**
Ferreira [7]	CSE-CIC-IDS2018, train 02-16, test 02-21 (unseen classes); MLP TPR 0.898 on all test attacks; LOIC-UDP TPR 0
Sakr [8]	CSE-CIC-IDS2018, 9 days; random forest F1 0.957 (random split), 0.239 (LOAO mean)
Verkerken [9]	CIC-IDS2017 random split: 41 of 47 unknown flows detected; CSE-CIC-IDS2018 Infiltration recall 78.38 %
Soltani [10]	CSE-CIC-IDS2018 packet bytes, one unknown label per run: its accuracy 35.47–72.16 %

The framework reaches a mean per-class F1 of 0.991 ± 0.004, against 0.979 ± 0.001 for raw CatBoost and 0.975 for the best re-run method. It is better than raw CatBoost in every seed, by 0.008 to 0.017 (paired bootstrap, 95 % intervals for seeds 3, 4 and 5: [0.006, 0.010], [0.015, 0.020], [0.009, 0.013]). With the detector, the F1 drops only to 0.990 ± 0.004; the detector labels about 1 % of the Benign hold-out flows as unknown attacks. Table 5 shows that the gain comes from Slowloris. Raw CatBoost is better on FTP, with 8 hold-out rows, and thus over all five classes (0.979 against 0.940).

The ablations in Table 4 locate the gain. Without the correction, the VAE classifier scores below raw CatBoost (0.965 against 0.979). The correction raises its Slowloris recall from 0.79 (not tabulated) to 0.98, and it lifts raw CatBoost to a mean F1 of 0.990 as well, a tie with the framework within the seed spread. Raw CatBoost with the correction still runs the VAE for the NLL and the latent cells, relabels more Benign flows and is slower than the framework. The gain therefore comes from the correction, which needs the VAE signals, rather than from the VAE as a feature compressor. In line with this, a Gaussian likelihood, no topological loss, or the latent mean only each changes the F1 by at most 0.003 when the correction is present, within the seed spread; without the correction, the topological loss lowers the F1. LightGBM and XGBoost in Stage 3 reach the F1 of CatBoost with a lower balanced accuracy. Giving Stage 3 both the 44 raw features and the seven VAE outputs does not close the gap before the correction (0.973 against 0.979) and matches the framework after it (0.991), but removes the FTP false alarms (FTP F1 0.98 against 0.74).

Inference Time. With the detector, the framework needs 0.050 ms per flow on one CPU thread, against 0.018 ms for raw CatBoost; its merged VAE and CatBoost graph takes 0.044 ms (0.046 ms as two graphs), the correction 0.00015 ms and the detector

Table 4. Hold-out mean per-class F1 (classes ≥30 rows) per seed and mean ± SD, balanced accuracy (all classes), Benign hold-out flows relabelled (sum over seeds; with the detector: by the correction and the detector), and CPU ms per flow (batch 1, one thread, seed 3). With the correction, the framework has a higher mean F1 than raw CatBoost and each re-run method.

Configuration	Seed 3	Seed 4	Seed 5	Mean F1	Bal. acc.	Relab.	ms/flow
Ours (with the correction)	0.9883	0.9949	0.9895	0.9909 ± 0.0036	0.9947 ± 0.0018	487	0.044
Without the correction	0.9583	0.9700	0.9667	0.9650 ± 0.0060	0.9569 ± 0.0051	–	–
Without the topological loss	0.9880	0.9931	0.9906	0.9905 ± 0.0026	0.9865 ± 0.0153	405	–
No topological loss or corr.	0.9742	0.9655	0.9772	0.9723 ± 0.0060	0.9590 ± 0.0255	–	–
Gaussian likelihood	0.9876	0.9908	0.9864	0.9883 ± 0.0023	0.9940 ± 0.0008	583	–
Latent mean only (5 inputs)	0.9883	0.9913	0.9875	0.9890 ± 0.0020	0.9946 ± 0.0013	614	–
LightGBM in Stage 3	0.9905	0.9935	0.9900	0.9913 ± 0.0019	0.9767 ± 0.0284	671	0.16
XGBoost in Stage 3	0.9910	0.9954	0.9905	0.9923 ± 0.0027	0.9441 ± 0.0015	614	0.058
Raw + VAE inputs (51) + corr.	0.9902	0.9893	0.9934	0.9910 ± 0.0022	0.9950 ± 0.0007	1209	–
Raw + VAE inputs (51)	0.9781	0.9634	0.9788	0.9734 ± 0.0087	0.9673 ± 0.0085	–	–
Raw CatBoost + correction	0.9911	0.9896	0.9907	0.9904 ± 0.0008	0.9955 ± 0.0003	1377	0.057
Raw CatBoost (44 features)	0.9800	0.9776	0.9785	0.9787 ± 0.0012	0.9739 ± 0.0014	–	0.018
Ours + detector	0.9868	0.9937	0.9883	0.9896 ± 0.0036	–	7674	0.050
Re-run methods (Table 3)							
Random forest [8]	0.9642	0.9643	0.9643	0.9643 ± 0.00004	0.9042 ± 0.00004	–	0.0040
MLP [7]	0.9781	0.9555	0.9600	0.9645 ± 0.0120	0.9628 ± 0.0148	–	0.0031
XGBoost [31]	0.9640	0.9640	0.9640	0.9640 ± 0	0.9042 ± 0	–	0.0053
LightGBM [32]	0.9655	0.9677	0.9629	0.9654 ± 0.0024	0.9549 ± 0.0002	–	0.0059
IDAE-XG [1]	0.6959	0.6423	0.6425	0.6602 ± 0.0309	0.6621 ± 0.0215	–	0.016
CNN-1D [33]	0.9742	0.9780	0.9734	0.9752 ± 0.0024	0.9740 ± 0.0032	–	0.039
FlowTransformer [13]	0.8276	0.8279	0.8402	0.8319 ± 0.0072	0.8827 ± 0.0189	–	0.058
TabM [34]	0.9493	0.9510	0.9502	0.9502 ± 0.0008	0.9542 ± 0.0001	–	0.39
FT-Transformer [35]	0.9608	0.9606	0.9626	0.9613 ± 0.0011	0.9546 ± 0.0014	–	0.74
TabNet [36]	0.9546	0.9531	0.9632	0.9569 ± 0.0054	0.9594 ± 0.0058	–	0.098

0.0058 ms. We timed each pipeline with a time in Table 4 as ONNX Runtime 1.31 graphs and C++ code on one thread of an Intel Core Ultra 9 275HX, one flow per call from the extracted features to the final label, over the 100,000 consecutive flows in the middle of day 02-21 (98,562 Benign burst and 1,438 attack flows). Each time is the median of 4 passes; a full repeat gave the same order of the 16 pipelines and a framework time 1.3 % higher.

Table 6 measures F1 and cost with the k most important features of Stage 1 (seed 3; $k = 64$: all 79 columns minus the 15 dropped); for $k > 10$, our F1 stays within 0.9870–0.9908, and $k \in \{23, 40, 64\}$ score above the $k = 44$ of the 95 % rule.

4.3 Unseen Attack Families

A deployed IDS will meet attack families that no stage has seen. Table 7 tests this in two ways: the detector alone on further capture days, each left out of its training together with its families, and the framework with the detector on the leave-one-attack-out folds. The detector mainly finds families whose flows arrive as bursts of novel flows. For LOIC-HTTP, it flags 99.01 % of the request flows (3 forward and 4 backward packets) but only 0.28 % of the connection attempts that receive no reply, and Infiltration stays mostly undetected. Its Benign FPR is above the 1 % target on 02-28 (2.42 %) and in all folds. The VAE inputs matter: a CatBoost detector on the 44 raw features, trained on the same rows with the same cut-off but without monotone constraints, flags on average only 4.17 %

Table 5. Per-class hold-out results, mean over seeds 3–5, of the framework and raw CatBoost (top), and the confusion matrix of the framework, seed 3 (bottom; rows: true class)

		Ours (with correction)			Raw CatBoost (44 features)		
Class	Rows	Precision	Recall	F1	Precision	Recall	F1
Benign	230,968	0.9998	0.9978	0.9988	0.9989	0.9981	0.9985
DoS GoldenEye	8,267	0.9964	0.9992	0.9978	0.9983	0.9998	0.9990
DoS Slowloris	1,933	0.9801	0.9767	0.9784	0.9929	0.8724	0.9288
FTP-BruteForce	8	0.6218	1.0000	0.7360	0.9630	1.0000	0.9804
SSH-BruteForce	18,807	0.9775	1.0000	0.9886	0.9777	0.9995	0.9884
Balanced accuracy		0.9947 ± 0.0018			0.9739 ± 0.0014		
Macro ROC-AUC (one vs rest, classifier)		0.9993 ± 0.0005			0.9995 ± 0.0001		
Macro PR-AUC (one vs rest, classifier)		0.9689 ± 0.0219			0.9924 ± 0.0011		

True \ predicted	Benign	GoldenEye	Slowloris	FTP	SSH
Benign	230,374	34	56	0	504
DoS GoldenEye	8	8,259	0	0	0
DoS Slowloris	59	0	1,874	0	0
FTP-BruteForce	0	0	0	8	0
SSH-BruteForce	1	0	0	0	18,806

Table 6. Feature-count ablation (seed 3): hold-out mean per-class F1, VAE training time in s (one VAE at a time on the GPU) and inference time in ms per flow (batch 1, one thread, no detector, as Table 4)

	F1				ms per flow	
k	Ours	Raw	Raw + corr.	VAE (s)	Ours	Raw
10	0.9738	0.9815	0.9881	141	0.046	0.017
15	0.9877	0.9792	0.9886	158	0.072	0.019
23	0.9908	0.9782	0.9895	273	0.051	0.015
30	0.9870	0.9799	0.9908	357	0.054	0.024
40	0.9902	0.9813	0.9911	302	0.061	0.019
44	0.9883	0.9800	0.9911	317	0.044	0.018
64	0.9908	0.9805	0.9906	392	0.057	0.016

(depth 6) and 4.81 % (depth 2) of the flows of each left-out family, against 30.86 % for ours (mean of Table 7a), so the labelled further days and the tree depth alone do not explain the detection. Averaged over day–family pairs, the framework with the detector detects 31.28 % of the flows at a mean daily Benign FPR of 1.16 %; its J, the mean over day–family pairs of the detection rate minus the FPR of that day, is 30.27.

Table 8 compares all methods by Youden's J. With the detector, the framework has the highest mean J on the left-out days and in the folds, while its classifier alone scores like most re-run methods (J 0.57 on the left-out days). The test day shows the limit of the burst signal: 99.4 % of its Benign flows belong to one burst inside the HOIC window. The framework detects 81.35 % of the LOIC-UDP flows, but also flags 66.77 % of this Benign burst, which gets higher detector scores than the HOIC flows. Three re-run methods have a higher J on LOIC-UDP, and no method has a positive J on HOIC.

Table 7. Attack families absent from all training labels, in %, mean ± SD over seeds 3–5: (a) left-out days, DR and Benign FPR of the detector alone; (b) leave-one-attack-out folds, DR on days 02-14 and 02-15 after each step, and hold-out Benign FPR; with raw CatBoost as classifier (same correction and detector), the mean final DR is 0.01 to 1.67 points higher per fold.

(a) Left-out days

Day	Family	DR	FPR
02-28	Infiltration	2.34±1.83	2.42±0.23
02-20	LOIC-HTTP	50.06±0.12	0.91±0.16
02-22	Web brute force	30.84±21.21	0.61±0.06
	SQL injection	13.04±7.53	
	XSS	48.44±17.61	
03-01	Infiltration	3.03±0.71	0.68±0.01
03-02	Bot	68.24±2.06	0.68±0.04

(b) Leave-one-attack-out folds

Removed class	Classifier	+ corr.	+ det.	FPR
SSH-BruteForce	0.0007±0.001	0.0007±0.001	95.16±5.02	2.19±1.21
DoS GoldenEye	0.11±0.01	11.96±1.40	58.19±19.27	1.68±0.92
DoS Slowloris	53.01±13.28	71.09±2.01	87.43±8.32	2.03±0.97

Table 8. Unseen attack families, in %, mean over seeds 3–5: Youden's J is the detection rate (DR) minus the Benign false-positive rate (FPR); a re-run method flags a flow whose argmax class is not Benign. Left-out days: mean over day and family; folds: mean over the three folds. Test day 02-21: DR, Benign FPR outside and inside the burst, and J against all Benign flows.

	J		Test day					
	Left-out days	Folds	DR UDP	DR HOIC	FPR out	FPR burst	J UDP	J HOIC
Ours, classifier	0.57	17.59	0	0.002	0.83	0.06	−0.07	−0.06
Ours + detector	30.27	78.29	81.35	0.01	15.42	66.77	14.88	−66.46
Random forest [8]	0.03	15.81	0	0	0.27	0.0003	−0.002	−0.002
MLP [7]	1.60	18.99	0	0	0.72	4.05	−4.03	−4.03
XGBoost [31]	−0.01	23.90	0	0	0.38	0	−0.002	−0.002
LightGBM [32]	−0.01	5.36	0.15	0	0.35	0.00009	0.15	−0.002
IDAE-XG [1]	24.57	36.52	99.81	32.17	68.17	67.64	32.16	−35.48
CNN-1D [33]	0.60	16.56	0.02	0	0.49	20.57	−20.43	−20.45
FlowTransformer [13]	8.33	8.02	93.29	17.32	38.35	68.57	24.90	−51.07
TabM [34]	0.02	16.84	63.91	0	1.36	0.57	63.34	−0.57
FT-Transformer [35]	0.31	20.85	6.61	0	0.64	0.94	5.67	−0.94
TabNet [36]	0.07	16.94	0	0.23	0.53	25.01	−24.87	−24.64

5 Discussion

The results support the design idea: the VAE is most useful through its reconstruction error and latent space, which let the correction recover known attacks in a new form and the detector find new families that arrive in bursts, where seen-class classifiers can fail [7,8]. They also explain the failures: unusual Benign bursts look like attacks, attacks without bursts give no signal, and the correction assumes one dominant attack at a time.

Limits. The evidence comes from one dataset with known construction errors and label faults [4,29] and from three seeds. The detector uses attack labels of other days that the re-run methods lack; open-set detectors remain to be compared.

6 Conclusion

We proposed a hybrid VAE-CatBoost framework in which a robust VAE serves a classifier and two streaming components. On CSE-CIC-IDS2018 with a time-ordered hold-

out, the framework improves the mean per-class F1 of the four larger classes over raw CatBoost and ten re-run methods, and with the detector, trained on attacks of other days, it has the highest Youden's J on left-out days and folds. The ablations show where the value of the VAE lies: its reconstruction error and latent cells drive the correction, which gives raw-feature and VAE-input classifiers the same F1, and the detector, which finds about seven times more flows of unseen families than a detector on raw features. Future work should separate unusual Benign bursts from attacks.

Use of AI Tools. Claude Code was used for the implementation of the code and for finding bugs.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Kang, Y., Tan, M., Lin, D., et al.: Intrusion detection model based on autoencoder and XGBoost. Journal of Physics: Conference Series **2171**(1), 012053 (2022)
2. Chen, Y., Zheng, X., Wang, N.: Construction of VAE-GRU-XGBoost intrusion detection model for network security. PLOS ONE **20**(6), e0326205 (2025)
3. Arp, D., Quiring, E., Pendlebury, F., et al.: Dos and don'ts of machine learning in computer security. In: USENIX Security. pp. 3971–3988 (2022)
4. Engelen, G., Rimmer, V., Joosen, W.: Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. In: IEEE Security and Privacy Workshops (SPW). pp. 7–12 (2021)
5. D'hooge, L., Verkerken, M., Volckaert, B., et al.: Establishing the contaminating effect of metadata feature inclusion in machine-learned network intrusion detection models. In: DIMVA. LNCS, vol. 13358, pp. 23–41. Springer (2022)
6. Flood, R., Engelen, G., Aspinall, D., et al.: Bad design smells in benchmark NIDS datasets. In: IEEE EuroS&P. pp. 658–675 (2024)
7. Ferreira, P., Antunes, M.: Benchmarking bioinspired machine learning algorithms with CSE-CIC-IDS2018 network intrusions dataset. In: 26th Portuguese Conference on Pattern Recognition (RECPAD) (2020)
8. Sakr, H.A., El-Douh, A.A., Lapina, M., et al.: Explainable and deployment-aware zero-day intrusion detection for cloud-level backend and management ecosystems in EV/V2X cyber-physical systems. Computers **15**(9), 599 (2026)
9. Verkerken, M., D'hooge, L., Sudyana, D., et al.: A novel multi-stage approach for hierarchical intrusion detection. IEEE Transactions on Network and Service Management **20**(3), 3915–3929 (2023)
10. Soltani, M., Ousat, B., Jafari Siavoshani, M., et al.: An adaptable deep learning-based intrusion detection system to zero-day attacks. arXiv:2108.09199 (2021)
11. Mirsky, Y., Doitshman, T., Elovici, Y., et al.: Kitsune: An ensemble of autoencoders for online network intrusion detection. In: Network and Distributed System Security Symposium (NDSS) (2018)
12. Bhatia, S., Hooi, B., Yoon, M., et al.: MIDAS: Microcluster-based detector of anomalies in edge streams. In: AAAI. vol. 34, pp. 3242–3249 (2020)
13. Manocchio, L.D., Layeghy, S., Lo, W.W., et al.: FlowTransformer: A transformer framework for flow-based network intrusion detection systems. Expert Systems with Applications **241**, 122564 (2024)
14. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems. vol. 30, pp. 4765–4774 (2017)

15. Kingma, D.P., Welling, M.: Auto-encoding variational Bayes. In: ICLR (2014)
16. Habibi Lashkari, A.: CICFlowMeter: source code, BasicFlow.java. https://github.com/ahlashkari/CICFlowMeter, last accessed 2026/10/18
17. Wu, M., Goodman, N.: Multimodal generative models for scalable weakly-supervised learning. In: Advances in Neural Information Processing Systems. vol. 31, pp. 5575–5585 (2018)
18. Ma, C., Tschiatschek, S., Palla, K., et al.: EDDI: Efficient dynamic discovery of high-value information with partial VAE. In: ICML. PMLR, vol. 97, pp. 4234–4243 (2019)
19. Ivanov, O., Figurnov, M., Vetrov, D.: Variational autoencoder with arbitrary conditioning. In: ICLR (2019)
20. Takahashi, H., Iwata, T., Yamanaka, Y., et al.: Student-t variational autoencoder for robust density estimation. In: IJCAI. pp. 2696–2702 (2018)
21. Rybkin, O., Daniilidis, K., Levine, S.: Simple and effective VAE training with calibrated decoders. In: ICML. PMLR, vol. 139, pp. 9179–9189 (2021)
22. Moor, M., Horn, M., Rieck, B., et al.: Topological autoencoders. In: ICML. PMLR, vol. 119, pp. 7045–7054 (2020)
23. Sinha, S., Dieng, A.B.: Consistency regularization for variational auto-encoders. In: Advances in Neural Information Processing Systems. vol. 34, pp. 12943–12954 (2021)
24. Prokhorenkova, L., Gusev, G., Vorobev, A., et al.: CatBoost: Unbiased boosting with categorical features. In: Advances in Neural Information Processing Systems. vol. 31, pp. 6638–6648 (2018)
25. Akiba, T., Sano, S., Yanase, T., et al.: Optuna: A next-generation hyperparameter optimization framework. In: ACM SIGKDD (KDD). pp. 2623–2631 (2019)
26. Íncer Romeo, Í., Theodorides, M., Afroz, S., et al.: Adversarially robust malware detection using monotonic classification. In: ACM International Workshop on Security and Privacy Analytics (IWSPA). pp. 54–63 (2018)
27. Canadian Institute for Cybersecurity: CSE-CIC-IDS2018 on AWS. https://www.unb.ca/cic/datasets/ids-2018.html, last accessed 2026/10/15
28. Sharafaldin, I., Habibi Lashkari, A., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP). pp. 108–116. SCITEPRESS (2018)
29. Liu, L., Engelen, G., Lynar, T., et al.: Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018. In: IEEE Conference on Communications and Network Security (CNS). pp. 254–262 (2022)
30. Koehn, P.: Statistical significance tests for machine translation evaluation. In: EMNLP. pp. 388–395 (2004)
31. Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: ACM SIGKDD (KDD). pp. 785–794 (2016)
32. Ke, G., Meng, Q., Finley, T., et al.: LightGBM: A highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems. vol. 30, pp. 3146–3154 (2017)
33. Donoso, O.E., Pacheco, M.A., Vargas, J.P., et al.: Autoencoder versus CNN-1D for network traffic intrusion detection: A reproducible comparative evaluation with per-class analysis on CSE-CIC-IDS2018. Journal of Cybersecurity and Privacy **6**(5), 183 (2026)
34. Gorishniy, Y., Kotelnikov, A., Babenko, A.: TabM: Advancing tabular deep learning with parameter-efficient ensembling. In: ICLR (2025)
35. Gorishniy, Y., Rubachev, I., Khrulkov, V., et al.: Revisiting deep learning models for tabular data. In: Advances in Neural Information Processing Systems. vol. 34, pp. 18932–18943 (2021)
36. Zegarra Rodríguez, D., Okey, O.D., Maidin, S.S., et al.: Attentive transformer deep learning algorithm for intrusion detection on IoT systems using automatic Xplainable feature selection. PLOS ONE **18**(10), e0286652 (2023)